

Analyse Méthodologique du Projet

Algorithmique et Complexité

Problème d'Ordonnancement $P, m \parallel L_{\max}$

MORLET Hugo, CASTRO Alexis, SABATIER Guillaume, RAHALI Linda, BENZAADI Naël

9 décembre 2025

Table des matières

1	Introduction	2
2	Question 1 : Ratio de Performance de l'Algorithme de Jackson	2
3	Question 2 : Programmation Dynamique	4
4	Question 3 : Schéma d'Approximation S (FPTAS)	4
4.1	Algorithme pour schéma d'approximation S (FPTAS)	5
4.2	Preuve : S est un FPTAS (pour m fixé)	5
4.2.1	Garantie de la qualité (erreur bornée par ϵ)	5
4.2.2	Garantie de la complexité (polynomiale)	7
4.3	Difficultés si m devient variable	8
5	Question 4 : Organisation des opérations de transport	8
6	Question 5 : Implémentation et Résultats	10
6.1	Protocole Expérimental et Jeu de Données	10
6.1.1	Jeu de données généré (Pool de 20 tâches)	10
6.2	Illustration de la Réduction FPTAS	10
6.3	Résultats Expérimentaux	11
6.4	Analyse	11
6.4.1	Limites de l'Approche Exacte	11
6.4.2	Efficacité du FPTAS	11
7	Conclusion	11

1 Introduction

L'objectif de ce projet est de résoudre le problème d'ordonnancement $P, m || L_{\max}$, qui consiste à minimiser la latence maximale de N tâches indépendantes sur m machines identiques, sans préemption.

Pour ce faire, nous étudions et implémentons en langage C trois approches distinctes :

- L'heuristique de **Jackson** (basée sur le tri des latences).
- Un algorithme exact de **programmation dynamique**.
- Un schéma d'approximation polynomial (**FPTAS**).

L'analyse théorique de leur complexité, couplée aux résultats expérimentaux, nous permettra de déterminer l'algorithme le plus performant pour ce problème de minimisation.

2 Question 1 : Ratio de Performance de l'Algorithme de Jackson

Définition du Problème et de l'Heuristique

Le problème considéré est le $P, m || L_{\max}$. Nous avons une instance (I) composée de :

- N tâches indépendantes.
- Pour chaque tâche i , une durée opératoire p_i et une durée de latence (ou "queue") q_i .
- m machines parallèles identiques.

L'objectif est de minimiser le critère $L_{\max} = \max_i (C_i + q_i)$, où C_i est la date de fin de la tâche i .

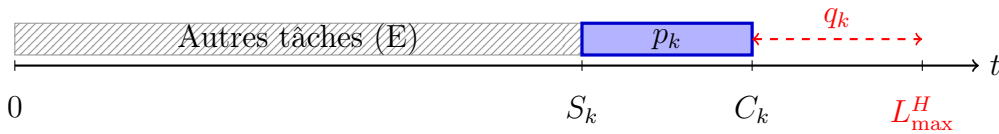
Nous analysons l'heuristique H connue sous le nom d'Algorithme de Jackson. Cet algorithme est un ordonnancement de liste qui fonctionne comme suit :

1. Trier les N tâches par ordre décroissant de leur latence q_i .
2. Dès qu'une machine se libère, on lui affecte la prochaine tâche disponible sur la liste.

Notre objectif est de trouver le ratio de performance $\rho = L^H / L^*$, où L^H est la solution de l'heuristique et L^* est la solution optimale.

Démonstration du ratio

1. Définition de la tâche critique (k) Supposons que k est la tâche qui nous donne $L_{\max}^H$ tel que : $C_k = S_k + p_k$ avec S_k la date de début de k .



L'objectif est défini par :

$$L_{\max}^H = S_k + p_k + q_k$$

2. Hypothèses sur l'ensemble E Supposons que E est l'ensemble des tâches qui s'exécutent avant S_k .

- **Hypothèse 1 (Propriété de Jackson)** : Comme on suit l'ordre de Jackson (par q_i décroissants), toute tâche j dans E a une priorité plus élevée ou égale à k , donc : $\forall j \in E, q_j \geq q_k$.
- **Hypothèse 2** : Toutes les machines étaient occupées avant S_k (sinon k aurait commencé plus tôt).

3. Borne Supérieure (Solution H) De l'Hypothèse 2, on déduit que le volume de travail $m \cdot S_k$ est inférieur ou égal à la somme des durées des tâches prioritaires (noté $\mathcal{J} = E \cup \{k\}$) moins p_k :

$$\begin{aligned} m \cdot S_k &\leq \sum_{j \in \mathcal{J}} p_j - p_k \\ \implies S_k &\leq \frac{1}{m} \sum_{j \in \mathcal{J}} p_j - \frac{p_k}{m} \end{aligned}$$

En substituant S_k dans l'équation de $L_{\max}^H$, on obtient :

$$L_{\max}^H = S_k + p_k + q_k \leq \left(\frac{1}{m} \sum_{j \in \mathcal{J}} p_j - \frac{p_k}{m} \right) + p_k + q_k$$

En réorganisant les termes :

$$L_{\max}^H \leq \frac{1}{m} \sum_{j \in \mathcal{J}} p_j + q_k + \left(1 - \frac{1}{m} \right) p_k \quad (*)$$

4. Borne Inférieure (Solution $L_{\max}^*$) La solution optimale $L_{\max}^*$ est au moins aussi grande que l'optimale sur le sous-ensemble $\mathcal{J}$. Cet optimal est contraint par la charge moyenne de $\mathcal{J}$ et la latence minimale de $\mathcal{J}$.

$$L_{\max}^* \geq L_{\max}^*(\mathcal{J}) \geq \frac{1}{m} \sum_{j \in \mathcal{J}} p_j + \min_{j \in \mathcal{J}} \{q_j\}$$

Grâce à l'Hypothèse 1 (Propriété de Jackson), on sait que $\min_{j \in \mathcal{J}} \{q_j\} = q_k$. On a donc la borne inférieure :

$$L_{\max}^* \geq \frac{1}{m} \sum_{j \in \mathcal{J}} p_j + q_k$$

5. Combinaison Finale En reprenant l'équation (*), on peut maintenant utiliser notre borne inférieure :

$$(*) \implies L_{\max}^H \leq \underbrace{\left(\frac{1}{m} \sum_{j \in \mathcal{J}} p_j + q_k \right)}_{\leq L_{\max}^*} + \left(1 - \frac{1}{m} \right) p_k$$

$$L_{\max}^H \leq L_{\max}^* + \left(1 - \frac{1}{m} \right) p_k$$

On utilise une dernière borne : $L_{\max}^* \geq p_k + q_k$, ce qui implique $p_k \leq L_{\max}^*$.

$$L_{\max}^H \leq L_{\max}^* + \left(1 - \frac{1}{m} \right) L_{\max}^*$$

$$L_{\max}^H \leq L_{\max}^* \left(1 + 1 - \frac{1}{m} \right)$$

$$L_{\max}^H \leq \left(2 - \frac{1}{m} \right) L_{\max}^*$$

3 Question 2 : Programmation Dynamique

On définit un état $e = [t_1, t_2, \dots, t_m, f]$ où :

- t_i : date de fin sur la machine M_i , pour $i = 1, \dots, m$
- f : valeur du $L_{\max}$ sur toutes les machines

Soit un état $e = [t_1, t_2, \dots, t_m, f]$ tel que $j \in \{1, 2, \dots, N\}$.

Au prochain saut, la tâche j sera affectée à une machine donnée. Ainsi, m états seront générés dans X_j .

Algorithme de Programmation Dynamique

Algorithm 1 Algorithme de Programmation Dynamique

```

1: Initialiser  $X_0 = \{[0, 0, \dots, 0, 0]\}$  ▷ État initial : charges nulles et  $f = 0$ 
2: for  $j = 1, 2, \dots, N$  do
3:   Initialiser  $X_j = \emptyset$ 
4:   for chaque état  $[t_1, \dots, t_m, f] \in X_{j-1}$  do
5:     for chaque machine  $i = 1, 2, \dots, m$  do
6:       Ajouter  $[t_1, t_2, \dots, t_i + p_j, \dots, t_m, \max\{f, t_i + p_j + q_j\}]$  à  $X_j$ 
7:     end for
8:   end for
9:   Supprimer  $X_{j-1}$ 
10:  Garder un état pour chaque valeur de  $t_i$  avec un  $f$  minimal
11: end for
12: return  $\min_{[t_1, \dots, t_m, f] \in X_N} f$ 

```

Complexité

La complexité de l'algorithme est :

$$N \cdot \left(\sum_{j=1}^N |X_{j-1}| \right) \cdot m = N \cdot P^{m-1} \cdot m$$

où P représente la somme totale des durées des tâches.

Ainsi :

$$\text{Complexité} = O(N \cdot P^{m-1} \cdot m)$$

$\implies$ L'algorithme est polynomial en N (mais pseudo-polynomial en fonction de P).

4 Question 3 : Schéma d'Approximation S (FPTAS)

La Question 2 nous a donné un algorithme pour $P_m \parallel L_{\max}$. Sa complexité est $O(N \cdot P^{m-1} \cdot m)$, ce qui est **pseudo-polynomial** à cause de la dépendance en P .

Pour la Question 3, nous créons un FPTAS en utilisant la technique de **modification des données**. Le principe est de « raccourcir » les durées p_j et les latences q_j en une nouvelle instance I' , puis de résoudre cette instance I' (qui est « petite ») avec notre algorithme exact de la Question 2.

4.1 Algorithme pour schéma d'approximation S (FPTAS)

Algorithm 2 Schéma FPTAS par modification des données

- 1: **Entrées** : Instance I (avec p_j, q_j), erreur $\epsilon > 0$
 - 2: **Étape 1 : Calculer une borne inférieure (LB)**
 - 3: $LB = \max\{\max_j(p_j + q_j) ; \frac{1}{m} \sum p_j\}$
 - 4: **Étape 2 : Calculer le facteur d'échelle (δ)**
 - 5: $\delta = \frac{\epsilon \cdot LB}{N + 1}$ ▷ Facteur $N + 1$ pour N durées p_j et 1 latence q_k
 - 6: **Étape 3 : Créer l'instance modifiée I'**
 - 7: **for** $j = 1$ **à** N **do**
 - 8: $p'_j = \lfloor \frac{p_j}{\delta} \rfloor$
 - 9: $q'_j = \lfloor \frac{q_j}{\delta} \rfloor$ ▷ Les latences sont **aussi** modifiées
 - 10: **end for**
 - 11: **Étape 4 : Résoudre**
 - 12: $S' = \text{Algo_Q2}(I')$ ▷ Algorithme exact de la Q2 sur I'
 - 13: **Étape 5 : Retourner S'** ▷ Solution optimale de $I' \approx$ solution de I
-

4.2 Preuve : S est un FPTAS (pour m fixé)

Pour être un FPTAS, S doit respecter deux conditions :

1. **Garantie de qualité** : erreur bornée par ϵ
2. **Garantie de complexité** : polynomiale en la taille de l'entrée et en $1/\epsilon$

4.2.1 Garantie de la qualité (erreur bornée par ϵ)

Nous devons montrer que $L_{\max}(S') \leq (1 + \epsilon)L_{\max}^*$.

Notations complémentaires

- I' : l'instance modifiée avec $p'_j = \lfloor p_j/\delta \rfloor$ et $q'_j = \lfloor q_j/\delta \rfloor$
- S^* : la solution optimale pour l'instance originale I , de valeur $L_{\max}^* = L_{\max}(S^*, I)$
- S' : notre solution, qui est optimale pour l'instance modifiée I'

1. Erreur d'arrondi

Par définition de la partie entière $\left(p'_j \leq \frac{p_j}{\delta} < p'_j + 1\right)$, on a :

- $p_j < \delta \cdot p'_j + \delta$
- $q_j < \delta \cdot q'_j + \delta$
- Et aussi :
- $\delta \cdot p'_j \leq p_j$
- $\delta \cdot q'_j \leq q_j$

2. Relation entre S' et $L_{\max}^*$

La preuve repose sur une chaîne d'inégalités reliant la solution S' à l'optimal $L_{\max}^*$.

Étape 1 : Borner la valeur réelle de S' Soit k la tâche critique de S' sur l'instance originale I . Soit E l'ensemble des tâches la précédant sur sa machine (au plus $N - 1$ tâches).

$$L_{\max}(S', I) = \sum_{j \in E \cup \{k\}} p_j + q_k$$

On utilise l'erreur d'arrondi sur chaque terme :

$$\begin{aligned} L_{\max}(S', I) &< \sum_{j \in E \cup \{k\}} (\delta p'_j + \delta) + (\delta q'_k + \delta) \\ &< \delta \left(\sum_{j \in E \cup \{k\}} p'_j + q'_k \right) + \underbrace{(\delta + \dots + \delta)}_{|E \cup \{k\}| \text{ fois}} + \delta \end{aligned}$$

Dans le pire cas, $|E \cup \{k\}| = N$ (toutes les tâches sur une machine) :

$$L_{\max}(S', I) < \delta \left(\sum_{j \in E \cup \{k\}} p'_j + q'_k \right) + N\delta + \delta$$

Le terme entre parenthèses est $L_{\max}(S', I')$ (la valeur de S' sur l'instance I') :

$$L_{\max}(S', I) \leq \delta \cdot L_{\max}(S', I') + (N + 1)\delta$$

Étape 2 : Utiliser l'optimalité de S' S' est la solution optimale pour I' . Elle est donc meilleure (ou égale) à n'importe quelle autre solution évaluée sur I' , y compris S^* :

$$L_{\max}(S', I') \leq L_{\max}(S^*, I')$$

En combinant avec l'Étape 1 :

$$L_{\max}(S', I) \leq \delta \cdot L_{\max}(S^*, I') + (N + 1)\delta$$

Étape 3 : Borner S^* sur I' Considérons la solution optimale S^* (valeur $L_{\max}^*$) évaluée sur I' . Soit k^* sa tâche critique.

$$L_{\max}(S^*, I') = \sum_{j \in E^* \cup \{k^*\}} p'_j + q'_{k^*}$$

En utilisant $\delta p'_j \leq p_j$ et $\delta q'_{k^*} \leq q_{k^*}$:

$$\delta \cdot L_{\max}(S^*, I') = \delta \sum_{j \in E^* \cup \{k^*\}} p'_j + \delta q'_{k^*} \leq \sum_{j \in E^* \cup \{k^*\}} p_j + q_{k^*}$$

Le terme de droite est $L_{\max}(S^*, I)$, c'est-à-dire $L_{\max}^*$.

$$\delta \cdot L_{\max}(S^*, I') \leq L_{\max}^*$$

Synthèse des trois étapes Reprenons l'inégalité de l'Étape 2 :

$$\begin{aligned} L_{\max}(S', I) &\leq \delta \cdot L_{\max}(S^*, I') + (N + 1)\delta \\ &\leq L_{\max}^* + (N + 1)\delta \quad (\text{d'après l'Étape 3}) \end{aligned}$$

3. Substitution de δ

Nous avons défini $\delta = \frac{\epsilon \cdot LB}{N+1}$ au début de l'algorithme. L'inégalité précédente devient donc :

$$\begin{aligned} L_{\max}(S') &\leq L_{\max}^* + (N + 1) \cdot \left(\frac{\epsilon \cdot LB}{N + 1} \right) \\ L_{\max}(S') &\leq L_{\max}^* + \epsilon \cdot LB \end{aligned}$$

4. Garantie de qualité finale

Par définition, LB est une borne inférieure de l'optimal, donc $LB \leq L_{\max}^*$. En utilisant cette propriété :

$$\begin{aligned} L_{\max}(S') &\leq L_{\max}^* + \epsilon \cdot LB \\ &\leq L_{\max}^* + \epsilon \cdot L_{\max}^* \\ &\leq L_{\max}^* \cdot (1 + \epsilon) \end{aligned}$$

Conclusion : Notre solution S' vérifie :

$$\boxed{L_{\max}(S') \leq (1 + \epsilon) \cdot L_{\max}^*}$$

Cette stratégie (scaler p_j et q_j , puis utiliser Algo_Q2) garantit bien une solution à un facteur $(1 + \epsilon)$ de l'optimal.

4.2.2 Garantie de la complexité (polynomiale)

1. Nouvelle complexité

On lance l'algorithme de la Q2 (complexité $O(N \cdot m \cdot P^{m-1})$) sur la nouvelle instance I' . La complexité est donc :

$$O(N \cdot m \cdot (P')^{m-1})$$

où $P' = \sum_{j=1}^N p'_j$ est la somme des durées modifiées.

2. Majoration de P'

Calculons une borne supérieure pour P' :

$$\begin{aligned} P' &= \sum_{j=1}^N p'_j = \sum_{j=1}^N \left\lfloor \frac{p_j}{\delta} \right\rfloor \\ &\leq \sum_{j=1}^N \frac{p_j}{\delta} = \frac{1}{\delta} \sum_{j=1}^N p_j = \frac{P}{\delta} \end{aligned}$$

En remplaçant $\delta = \frac{\epsilon \cdot LB}{N+1}$:

$$P' \leq \frac{P}{\frac{\epsilon \cdot LB}{N+1}} = \frac{P \cdot (N+1)}{\epsilon \cdot LB}$$

Or, on sait que :

- $P = \sum_{j=1}^N p_j \leq N \cdot \max_j \{p_j\}$
 - $LB = \max_j \{p_j + q_j\} \geq \max_j \{p_j\}$
- Donc $\frac{P}{LB} \leq \frac{N \cdot \max_j \{p_j\}}{\max_j \{p_j\}} = N$

En reprenant notre majoration de P' :

$$P' \leq \frac{P}{LB} \cdot \frac{N+1}{\epsilon}$$

En utilisant le fait que $\frac{P}{LB} \leq N$, on obtient :

$$P' \leq N \cdot \frac{N+1}{\epsilon} = O\left(\frac{N^2}{\epsilon}\right)$$

3. Complexité finale

En injectant la majoration de P' dans la complexité de l'algorithme :

$$\begin{aligned}\text{Complexité} &= O(N \cdot m \cdot (P')^{m-1}) \\ &= O\left(N \cdot m \cdot \left(\frac{N^2}{\epsilon}\right)^{m-1}\right)\end{aligned}$$

4. Conclusion

La complexité devient alors un **polynôme en N et $1/\epsilon$** . Par exemple, pour $m = 3$:

$$O\left(N \cdot 3 \cdot \left(\frac{N^2}{\epsilon}\right)^2\right) = O\left(\frac{N^5}{\epsilon^2}\right)$$

Cette complexité est bien polynomiale en la taille de l'entrée (N) et en $1/\epsilon$.

Le schéma S est bien un FPTAS quand m est fixé.

4.3 Difficultés si m devient variable

Si m n'est plus une constante mais une variable, deux obstacles majeurs apparaissent :

1. Difficulté de complexité

La complexité $O\left(N \cdot m \cdot \left(\frac{N^2}{\epsilon}\right)^{m-1}\right)$ contient un m dans l'exposant. Elle devient donc **exponentielle en m** et n'est plus polynomiale en la taille de l'entrée.

Pour exemple, si m double, la complexité est multipliée par un facteur exponentiel :

$$\left(\frac{N^2}{\epsilon}\right)^{2m-1} \text{ est exponentiellement plus grand que } \left(\frac{N^2}{\epsilon}\right)^{m-1}$$

5 Question 4 : Organisation des opérations de transport

Définition du Problème et de l'Heuristique

Le problème considéré ici est le $P2 \mid r_i \mid C_{\max}$. Il consiste à ordonnancer les opérations de transport réalisées par 2 robots ($m = 2$) sur les N tâches précédentes qui nous admettons, auront déjà été ordonnancées sur les machines. Pour chaque tâche de transport i , nous définissons :

- Une date de disponibilité r_i : correspondant à la date de fin d'exécution de la tâche sur la machine (C_i calculée précédemment).
- Une durée de transport q_i : correspondant à la latence de la tâche.

L'objectif est de minimiser la date de fin du dernier transport : $C_{\max} = \max_i \{C'_i\}$. Ce problème est NP-difficile. Nous proposons donc une heuristique gloutonne basée sur la règle LPT (*Longest Processing Time*) : dès qu'un robot est libre, on lui affecte la tâche disponible la plus longue.

Algorithm 3 Heuristique LPT pour les Robots (Transport)

```
1: Entrées : Dates de disponibilité  $\{r_1, \dots, r_N\}$ , Durées transport  $\{q_1, \dots, q_N\}$ 
2: Initialiser  $L = \{1, 2, \dots, N\}$  ▷ Ensemble des tâches non transportées
3: Initialiser  $T_{R1} = 0, T_{R2} = 0$  ▷ Temps de fin des 2 robots
4: while  $L \neq \emptyset$  do
5:    $i \leftarrow$  indice du robot tel que  $T_{Ri} = \min(T_{R1}, T_{R2})$  ▷ Robot libre le plus tôt
6:    $t_{courant} \leftarrow T_{Ri}$ 
7:   Construire  $D = \{j \in L \mid r_j \leq t_{courant}\}$  ▷ Tâches disponibles
8:   if  $D \neq \emptyset$  then
9:     Choisir  $k \in D$  tel que  $q_k = \max_{j \in D}\{q_j\}$  ▷ Règle LPT
10:     $T_{Ri} \leftarrow t_{courant} + q_k$  ▷ Mise à jour du robot
11:    Supprimer la tâche  $k$  de l'ensemble  $L$ 
12:   else
13:     $T_{Ri} \leftarrow \min_{j \in L}\{r_j\}$  ▷ Attente : avance au prochain  $r_j$ 
14:   end if
15: end while
16: return  $\max(T_{R1}, T_{R2})$ 
```

Analyse de performance

L'algorithme proposé garantit un ratio d'approximation de 2. Soit $C_{\max}(LPT)$ la date de fin obtenue et $C_{\max}^*$ l'optimal. Soit k la dernière tâche à se terminer. On note S_k sa date de début et $C'_k = S_k + q_k = C_{\max}(LPT)$. On distingue deux cas :

Cas 1 : La tâche k commence dès sa disponibilité ($S_k = r_k$) On a directement :

$$C_{\max}(LPT) = r_k + q_k$$

Or, l'optimal ne peut faire mieux que la date de disponibilité plus la durée : $C_{\max}^* \geq r_k + q_k$.
Donc $C_{\max}(LPT) \leq C_{\max}^*$.

Cas 2 : La tâche k commence après sa disponibilité ($S_k > r_k$) C'est le cœur de la preuve. Si la tâche k ne commence pas dès qu'elle est disponible (r_k), c'est que les robots étaient occupés. Puisque k est disponible mais ne commence pas, les **2 robots travaillent à plein régime**.

- (a) **Volume de travail** : La quantité de travail effectuée durant l'intervalle est $2 \times (S_k - r_k)$. Ce volume est nécessairement inférieur à la somme totale des durées de transport (moins la tâche k elle-même). On a donc :

$$2 \cdot (S_k - r_k) < \sum_{i=1}^N q_i$$

- (b) **Borne sur S_k** : Cela nous donne :

$$S_k < r_k + \frac{1}{2} \sum_{i=1}^N q_i$$

- (c) **Calcul du Makespan** : En ajoutant q_k de chaque côté :

$$C_{\max}(LPT) = S_k + q_k < r_k + q_k + \frac{1}{2} \sum_{i=1}^N q_i$$

Conclusion sur le ratio Nous savons que la solution optimale est bornée par le chemin critique ($C_{\max}^* \geq r_k + q_k$) et la charge moyenne ($C_{\max}^* \geq \frac{1}{2} \sum q_i$). En injectant ces bornes :

$$C_{\max}(LPT) < C_{\max}^* + C_{\max}^* = 2 \cdot C_{\max}^*$$

Que l'on soit dans le Cas 1 ou le Cas 2, l'algorithme garantit donc bien :

$$\frac{C_{\max}(LPT)}{C_{\max}^*} \leq 2$$

6 Question 5 : Implémentation et Résultats

6.1 Protocole Expérimental et Jeu de Données

Les algorithmes ont été implémentés en langage C. Afin de garantir la reproductibilité et la cohérence des comparaisons, nous avons généré un pool unique de 20 tâches avec des durées opératoires p_j et des latences q_j tirées aléatoirement dans l'intervalle $[1, 500]$.

Pour chaque test de taille N (que ce soit $N = 10, 12$ ou 20), nous sélectionnons systématiquement les N premières tâches de ce pool.

6.1.1 Jeu de données généré (Pool de 20 tâches)

Le tableau suivant détaille les caractéristiques des tâches utilisées pour tous nos tests :

ID	p _j	q _j	ID	p _j	q _j	ID	p _j	q _j	ID	p _j	q _j
T1	20	255	T6	355	254	T11	250	187	T16	21	253
T2	355	376	T7	230	261	T12	460	393	T17	310	108
T3	251	343	T8	164	262	T13	268	100	T18	129	165
T4	430	87	T9	349	331	T14	38	41	T19	285	275
T5	264	381	T10	269	357	T15	271	341	T20	117	370

TABLE 1 – Données brutes : Pool de 20 tâches ($p_{\max} = 500$).

6.2 Illustration de la Réduction FPTAS

Le cœur du schéma d'approximation réside dans la transformation de l'instance originale I en une instance simplifiée I' . Prenons l'exemple du test critique ($N = 12, m = 3$).

- **Paramètres** : $\epsilon = 0.20$, Borne Inférieure $LB = 1132$.
- **Facteur d'échelle calculé** : $\delta = \frac{\epsilon \cdot LB}{N+1} = \frac{0.2 \times 1132}{13} \approx \mathbf{17.41}$.

Les durées sont divisées par δ puis arrondies ($\lfloor p_j / 17.41 \rfloor$). Le tableau ci-dessous montre l'effet drastique de cette réduction sur les 5 premières tâches :

Tâche	Originale (I)		Réduite (I')	
	p_j	q_j	$p'_j = \lfloor p_j / \delta \rfloor$	$q'_j = \lfloor q_j / \delta \rfloor$
T1	20	255	1	14
T2	355	376	20	21
T3	251	343	14	19
T4	430	87	24	4
T5	264	381	15	21

TABLE 2 – Exemple de réduction des données pour le cas (12, 3). Les valeurs manipulées par l'algorithme dynamique sont divisées par ≈ 17 , réduisant ainsi l'espace d'états.

6.3 Résultats Expérimentaux

Le tableau final compare les performances. Le gain de temps devient spectaculaire sur les instances difficiles (comme $N = 12, m = 3$) grâce à la réduction illustrée ci-dessus.

Config (N, m)	Jackson		DP Exact		FPTAS ($\epsilon = 0.20$)		
	L_{max}	Temps	L_{max}	Temps	L_{max}	Temps	Gain
(10, 2)	1517	< 0.01 s	1653	0.02 s	1665	< 0.01 s	x19
(10, 3)	1148	< 0.01 s	1139	5.13 s	1143	0.27 s	x18
(20, 2)	2691	< 0.01 s	2640	5.43 s	2642	0.02 s	x321
(12, 3)	1353	< 0.01 s	1431	3 min 58 s	1431	1.72 s	x138

TABLE 3 – Performance sur grandes instances ($p_{max} = 500$). Le temps explose pour l’approche exacte sur (12,3).

6.4 Analyse

6.4.1 Limites de l’Approche Exacte

Les tests sur 3 machines illustrent la complexité pseudo-polynomiale $O(N \cdot P^{m-1})$. Pour $N = 12$, le temps explose à 237 secondes (près de 4 minutes). L’ajout de 2 tâches (par rapport à $N = 10$) a multiplié le temps de calcul par 46, confirmant que l’approche exacte n’est pas viable pour de grandes instances.

6.4.2 Efficacité du FPTAS

- Le FPTAS démontre sa pertinence sur le cas critique (12, 3) :
- Vitesse : Accélération de **x138**.
 - Précision : Il retrouve exactement la même solution ($L_{max} = 1431$), prouvant que la réduction drastique des données ($\delta \approx 17.4$) n’a pas dégradé la qualité de la solution finale.

7 Conclusion

Ce projet nous a permis d’analyser le problème d’ordonnancement $P, m || L_{max}$ sous l’angle du compromis entre le temps de calcul et la qualité de la solution.

Nos expérimentations ont mis en évidence les limites physiques de l’approche exacte (Programmation Dynamique), dont la complexité explose dès que le nombre de machines ou la durée des tâches augmente. À l’inverse, le schéma d’approximation (FPTAS) s’est imposé comme la stratégie la plus viable pour des applications réelles : il contourne l’explosion combinatoire en offrant une solution quasi-optimale (erreur < 1%) en un temps négligeable.

L’extension du problème à la gestion de robots de transport (Question 4) a également enrichi notre approche en intégrant des contraintes de disponibilité réalistes.

En somme, ce travail illustre qu’en algorithmique industrielle, la recherche de l’optimum absolu est souvent moins pertinente que l’obtention rapide d’une solution maîtrisée, validant ainsi la puissance des méthodes d’approximation.